



بسم الله الرحمن الرحيم

Sudan University of Science and Technology

Faculty of Graduate Studies



Faculty of Computer Sciences & Information Technology

Enhancing Handwritten Text Recognition Using Transformer Models Derived from Large Language Models

(تعزيز التعرف علي النصوص اليدوية باستخدام نماذج المحولات المستمدة من
نماذج لغوية كبيرة)

A Thesis Submitted in Partial Fulfillment of the Requirements for

M.Sc. degree in Computer Science

By: Elzubier Alsayed Algali

Supervisors:

Dr. Waleed Mergani

January 2024

Declaration

Introductive

Dedication

Acknowledgement

Abstract

المستخلص

Table of Contents

List of figures

List of Tables

List of abbreviations

Chapter One

Introduction

1. CHAPTER I

1.1. Introduction:

In the vast realm of artificial intelligence, computer vision emerges as a beacon of innovation, striving to imbue machines with the ability to comprehend and interpret the visual intricacies of our world. With a primary focus on decoding visual data, such as images and videos, this field entails the development of sophisticated algorithms and techniques. These advancements empower computers to recognize objects, discern patterns, and contextualize the information they perceive. Amid the challenges faced in the realm of character recognition, deciphering handwritten characters stands out as a particularly intricate task. The diversity in writing styles further complicates the function of character recognition. Recent strides in this domain have been made possible by the remarkable progress of Convolutional Neural Networks (CNNs). Their prowess extends to image-based recognition, video analytics, and natural language processing, ushering in a new era of possibilities (Skala, V., Singh, T.P., Choudhury, T., Tomar, R. and Bashar, M.A. eds., Md. Abul Bashar Editors, 2022).

A pivotal application of computer vision lies in the preservation of cultural heritage through the recognition of handwritten text on historical documents. Recognizing the significance of this task, the last decade has witnessed a shift towards the adoption of Recurrent Neural Networks (RNNs) coupled with Connectionist Temporal Classification (CTC). This combination forms an end-to-end system capable of transcribing raw images containing entire lines of text. In recent Handwritten Text Recognition (HTR) contests, RNN-CTC methods have exhibited the lowest error rates, with multidimensional Long Short-Term Memory (MDLSTM) networks at their core, offering recurrent connections along each dimension in the data (Sahare, et al., 2018).

Notably, the importance of HTR transcends the boundaries of historical document preservation, finding applications in document digitization and even the healthcare sector. The interpretation of handwritten medical prescriptions becomes a critical task, where traditional approaches relying on RNNs or CNNs have showcased significant progress. However, the landscape is evolving with the advent of Large Language Models (LLMs) and transformer architectures. These models, renowned for their exceptional performance in natural language processing, present an exciting opportunity to revolutionize HTR and redefine the boundaries of what is achievable in this dynamic field.

1.2. Motivation:

Recent advances in deep learning and transformer-based approaches offer opportunities to improve the accuracy and efficiency of handwritten text recognition (HTR). By exploring the potential of these approaches to improve HTR, this research aims to support the development of more effective recognition tools for HTR, potentially revolutionizing the way the handwritten text is recognized.

Furthermore, this research is motivated by growing interest in the application of artificial intelligence (AI) in optical character recognition (OCR) technologies.

By investigating the use of these approaches in handwritten text recognition (HTR), this study aims to contribute to the growing literature on the application of AI in computer vision field and pave the way for more innovative and effective OCR tools in the future.

1.3. Problem Statement:

This research seeks to address the inherent challenges within Handwritten Text Recognition (HTR), particularly focusing on the limitations associated with current methodologies. The recognition of handwritten characters poses significant difficulties due to variations in size, shape, and writing styles among individuals. Traditional approaches, such as recurrent neural networks (RNNs) and convolutional neural networks (CNNs), while demonstrating progress, exhibit limitations in handling the intricacies of diverse handwriting styles.

1.4. Aims and Objectives:

1.4.1. Aims:

The aim of this research is to enhance handwritten text recognition by leveraging transformer models derived from large language models.

1.4.2. Objectives:

The specific objectives of this research are:

1. Literature Review and Methodology Investigation:

Conduct a comprehensive review of existing HTR methods, with a focus on traditional approaches and their limitations.

2. Adaptation of Transformer Architecture:

Refine transformer architectures for handwritten text recognition challenges by customizing attention mechanisms, positional encoding, and self-attention mechanisms to suit the unique spatial characteristics of handwriting.

3. Data Collection and Model Training:

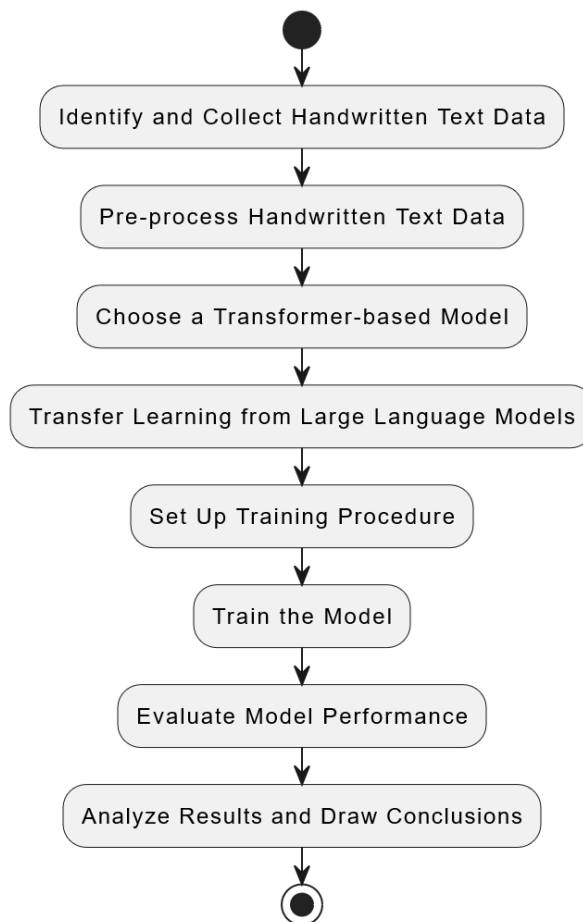
Assemble a diverse dataset of handwritten text samples, emphasizing variability in handwriting styles. Give specific attention to practical applications, such as medical prescriptions. Train transformer-based models on the compiled dataset, employing transfer learning from pre-trained Large Language Models (LLMs), and implement fine-tuning strategies to improve the model's capability in recognizing handwritten text.

4. Performance Assessment and Generalization:

Evaluate the performance of the proposed models using standard metrics like accuracy, precision, recall, and F1 score. Conduct comparative analyses with baseline models and traditional HTR approaches. Investigate the robustness and generalization capabilities of the proposed models across diverse handwriting styles.

1.5. Research Methodology:

This methodology outlines a comprehensive approach to advancing handwritten text recognition through the strategic integration of the IAM Handwriting Dataset, Transformer-based models, and transfer learning from Large Language Models. By meticulously addressing data selection, preprocessing, model training, and performance evaluation, this research methodology aims to enhance the understanding and application of sophisticated language models in the specialized context of handwritten text recognition.



1. Identify and Collect Handwritten Text Data

Selecting the "IAM Handwriting Dataset" is a judicious choice in alignment with the objectives of my handwritten text recognition research. I will ensure that this dataset incorporates a diverse range of handwriting samples, thereby fostering enhanced generalization capabilities within the model.

2. Pre-process Handwritten Text Data

Implement pre-processing steps such as normalization, data augmentation, and resizing to prepare the data for training.

3. Choose a Transformer-based Model

Selecting an open-source Large Language Model (LLM) such as GPT-3, BERT, GPT-2, XLNet, or a similar architecture is a strategic decision for my handwritten text recognition research. The next step involves determining the most suitable transformer model architecture among these options. Additionally, careful consideration will be given to any necessary modifications or the incorporation of additional layers tailored to the specific requirements of the handwritten text recognition task.

4. Transfer Learning from Large Language Models

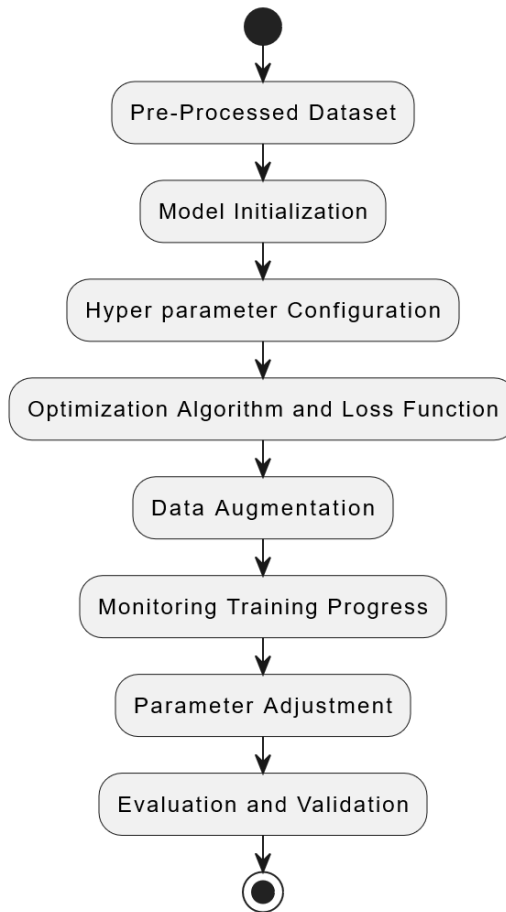
Leveraging transfer learning from large language models (LLMs) is a pivotal strategy in the context of my research on handwritten text recognition. This approach involves harnessing the pre-existing knowledge encoded within sophisticated language models such as GPT-3, BERT, GPT-2, XLNet, or similar architectures and adapting this knowledge to enhance the performance of my model on the specific task of recognizing handwritten text.

5. Set Up Training Procedure

Establishing the training procedure is a critical step in the development of a handwritten text recognition model. This involves defining various components, including hyper parameters, optimization algorithms, convergence criteria, and optionally, exploring parallel or distributed training methods to efficiently utilize available resources.

6. Train the Model

The training phase involves the implementation of the proposed transformer model on the pre-processed handwritten text dataset. The following steps outline the process:



a. **Pre-Processed Dataset**

Utilize the selected handwritten text dataset, which has undergone pre-processing steps to ensure uniformity, cleanliness, and compatibility with the transformer model.

b. **Model Initialization**

Initialize the transformer model architecture based on the chosen large language model (LLM), incorporating any modifications or additional layers necessary for handwritten text recognition.

c. **Hyper parameter Configuration**

Set hyper parameters such as learning rate, batch size, and the number of training epochs. These values should be determined based on experimentation and adjusted to optimize the convergence and accuracy of the model.

d. **Optimization Algorithm and Loss Function**

Implement the chosen optimization algorithm, such as Adam, along with an appropriate loss function tailored to the task of handwritten text recognition. Ensure that the loss function aligns with the objectives of the research.

e. **Data Augmentation**

Apply data augmentation techniques during training to introduce variability into the dataset, enhancing the model's ability to generalize to different handwriting styles.

f. **Monitoring Training Progress**

Implement logging and monitoring mechanisms to track key metrics, including training and validation loss, accuracy, and any other relevant performance indicators. Monitor convergence trends to determine when the model is sufficiently trained.

g. **Parameter Adjustment**

Regularly evaluate the model's performance on validation data and be prepared to adjust hyper parameters or other configurations as needed. This iterative process ensures continual refinement of the model throughout the training phase.

h. **Evaluation and Validation**

Periodically assess the model's performance on a separate validation set to ensure that it generalizes well to unseen data. Fine-tune parameters based on the observed results.

By systematically executing these steps, the training phase aims to empower the transformer model with the ability to effectively recognize handwritten text, leveraging insights derived from large language models for enhanced performance in the specific context of the research objectives.

7. Evaluate Model Performance

Define metrics for evaluating model performance, such as accuracy, precision, recall, and F1 score. Compare the results with baseline models or existing methods for benchmarking.

8. Analyze Results and Draw Conclusions

In the final stage of the research process, the analysis of results and drawing conclusions is pivotal to understanding the effectiveness of the proposed approach in enhancing handwritten text recognition using transformer models derived from large language models. The following steps outline how this phase can be conducted:

a) **Quantitative Results Presentation:**

Present the quantitative results obtained from the experiments, including metrics such as accuracy, precision, recall, F1 score, and any other relevant evaluation criteria. Use visualizations such as charts or graphs to highlight trends and patterns in the performance of the model over different training epochs.

b) **Qualitative Results Presentation:**

Provide qualitative insights by showcasing examples of handwritten text recognition outputs generated by the model. This can include side-by-side comparisons of the model's predictions against ground truth labels, emphasizing instances where the model excels or faces challenges.

c) **Comparison with Baseline Models:**

Compare the performance of the proposed transformer model with baseline models or traditional handwritten text recognition approaches. Highlight improvements or differences in accuracy and efficiency achieved through the novel approach.

1.6. Research Scope:

The research scope centers on optimizing Handwritten Text Recognition (HTR) specifically for English language handwritten text, leveraging a pre-trained model trained on the "IAM Handwriting" dataset its contains around 1,539 text pages. Each page typically includes multiple lines of handwritten text, contributing to a diverse set of samples for training and evaluation purposes.

1.7. Research Planning:

- Weeks 1-4: Literature Review and Model Design
- Weeks 5-7: Dataset Collection and Preprocessing
- Weeks 8-10: Model Training and Fine-Tuning
- Weeks 11-12: Evaluation and Analysis, Thesis Writing

1.8. The Thesis Organization

This research organized as following:

Chapter I: contains the research problem statement and objectives.

Chapter II: discusses the literature review and related work.

Chapter III: describes the research methodology and the implementation of the techniques used

Chapter IV: presents the Experiments and results. Finally.

Chapter V: concludes this research and presents Recommendations for future works.

Chapter Two

Literature Review

2.CHAPTER II

2.1. Introduction

Recent advancements in handwritten text recognition (HTR) have witnessed a paradigm shift, driven by the integration of deep learning methodologies. Notably, the convergence of transformer-based models, pre-trained architectures, and innovative training strategies has propelled HTR into new realms of accuracy and efficiency. This literature review explores diverse approaches, ranging from fine-tuning vision encoder-decoder transformers for historical document transcription to the development of novel architectures like TrOCR and Handwriting Transformers. The synthesis of these studies not only underscores the transformative potential of deep learning in HTR but also highlights the ongoing evolution towards more effective, efficient, and versatile recognition systems.

2.2. Problem Background

In the vast realm of artificial intelligence, the evolution of computer vision emerges as a captivating journey, a quest to empower machines with the extraordinary ability to decipher the intricate language of visuals. At the heart of this narrative lies a profound challenge – the recognition of handwritten characters, a task made all the more complex by the kaleidoscope of diverse writing styles. In this unfolding saga, Convolutional Neural Networks (CNNs) have played a pivotal role, advancing the frontier of image-based recognition. Yet, recent chapters in the story of Handwritten Text Recognition (HTR) reveal a shift towards a new protagonist – Recurrent Neural Networks (RNNs) coupled with Connectionist Temporal Classification (CTC), orchestrating the transcription of entire lines of text.

Beyond the realms of character recognition, our tale weaves through the rich tapestry of cultural heritage preservation, historical document analysis, and the digitization of documents, each page holding a story waiting to be told. In the corridors of healthcare, the narrative takes a critical turn as handwritten medical prescriptions demand interpretation. However, the journey encounters challenges – the limitations of traditional approaches relying on RNNs and CNNs to navigate the diverse landscape of handwriting styles.

Cue the turning point – a surge in deep learning and the advent of transformer-based approaches. Our story is now infused with the promise of improvement, as we seize the opportunities presented by Large Language Models (LLMs) and transformer architectures, celebrated champions in the arena of natural language processing. The stage is set for a redefinition of boundaries in the dynamic field of HTR.

The plot thickens as we confront the problem statement – an exploration of the inherent challenges in HTR, a focused gaze on the limitations of existing methodologies in decoding the unique variations in size, shape, and styles of handwritten characters. The traditional actors, RNNs and CNNs, have made strides but struggle with the nuances of diverse handwriting styles.

Our hero emerges with a clear aim – to enhance handwritten text recognition through the strategic leverage of transformer models born from large language models. The narrative unfolds with objectives that read like chapters – a comprehensive literature review to set the stage, the adaptation of transformer architecture to confront HTR challenges head-on, a quest for diverse data and rigorous model training, with a spotlight on practical applications like medical prescriptions. The climax arrives with a performance assessment, an evaluation of our hero's journey across diverse handwriting styles, a testament to its versatility.

In essence, our story is an epic, a journey that seeks to bridge the gap between the familiar terrain of traditional HTR methods and the uncharted territories of the transformative potential offered by transformer models. It is a story of addressing nuanced challenges, of unlocking the secrets encoded in handwritten characters, and ultimately, a tale of paving the way for more effective and versatile HTR tools in the ever-evolving saga of artificial intelligence.

2.3. Images Processing

Embarking on the realm of pixels and patterns, image processing emerges as a transformative art. This method, a digital alchemy, orchestrates operations to breathe life into images, birthing enhanced versions or revealing hidden insights. In the technological symphony, image processing stands tall among swiftly evolving disciplines. It weaves its

magic as a core research domain, entwining engineering and computer science in its tapestry of possibilities.

Delving deeper, image processing is the digital metamorphosis of visuals, a conversion of images into a binary language that unveils their secrets. This method, a signal dance, transforms pixels into a canvas where operations unfold, crafting a narrative of enhanced visuals or extracted intelligence. As (Covington, Michael A, 2009) echoes, image processing is the art of translating pixels into a digital language, a symphony where inputs are images, and outputs are reborn images or their inherent characteristics. It's a dance of signals, a harmonious interplay between what the eye sees and what the mind deciphers. In essence, image processing is more than a method – it's a voyage into the heart of visuals, unraveling their mysteries and enhancing their tales through the wizardry of algorithms and signals (Naveenkumar, , M. and V. Ayyasamy., 2016).

2.4. Optical Character Recognition (OCR)

Optical character recognition refers to the branch of computer science that involves reading text from paper and translating the images into a form that the computer can manipulate (for example, into ASCII codes).

An OCR system enables you to take a book or a magazine article, feed it directly into an electronic computer file, and then edit the file using a word processor. All OCR systems include an optical scanner for reading text, and sophisticated software for analyzing images. Most OCR systems use a combination of hardware (specialized circuit boards) and software to recognize characters, although some inexpensive systems do it entirely through software. Advanced OCR systems can read text in large variety of fonts, but they still have difficulty with handwritten text. It is the mechanical or electronic translation of scanned images of handwritten, typewritten or printed text into machine-encoded text. It is widely used to convert books and documents into electronic files, to computerize a record-keeping system in an office, or to publish the text on a website.

OCR makes it possible to edit the text, search for a word or phrase, store it more compactly, display or print a copy free of scanning artifacts, and apply techniques such as machine translation, text-to-speech and text mining to it. OCR is a field of research in pattern recognition, artificial intelligence and computer vision.

OCR systems require calibration to read a specific font; early versions needed to be programmed with images of each character, and worked on one font at a time. "Intelligent" systems with a high degree of recognition accuracy for most fonts are now common. Some systems are capable of reproducing formatted output that closely approximates the original scanned page including images, columns and other non-textual components.

Optical character recognition (OCR) is one of the most popular areas of research in pattern recognition because of its immense application potential. However, most of the available methods deal with the recognition of the Roman script and some of the oriental scripts like Kanji, Kana, etc. The purpose of this OCR system is to take handwritten English characters as input, process the character, train the neural network algorithm, to recognize the pattern and modify the character to a beautified version of the input. This work is restricted to English characters and numerals only. It is also helpful in recognizing special characters. It can be further developed to recognize the characters of different languages.

One of the primary means by which computers are endowed with humanlike abilities is through the use of a neural network. Neural networks are particularly useful for solving problems that cannot be expressed as a series of steps, such as recognizing patterns, classifying them into groups, series prediction and data mining (Isheawy, N.A.M. and Hasan, H., 2015).

2.5. Handwritten Text Recognition.

In the expansive domain of artificial intelligence and computer vision, the scientific exploration of Handwritten Text Recognition (HTR) unfurls as a multifaceted journey, intricately woven with challenges and technological evolution.

The scientific inquiry commences with the underpinning of computer vision, a discipline committed to imparting machines with the perceptual acuity to comprehend and interpret visual data. At the forefront of this exploration lies the nuanced task of recognizing handwritten characters, encapsulating the diverse spectrum of human writing styles.

Convolutional Neural Networks (CNNs) emerge as stalwarts, advancing image-based recognition and laying the foundation for decoding visual data. Recent forays into HTR navigate the complexities of character recognition using Recurrent Neural Networks (RNNs) coupled with Connectionist Temporal Classification (CTC), particularly adept at transcribing entire lines of handwritten text (Sahare, et al., 2018)

Beyond the realm of mere character recognition, the significance of HTR extends into the preservation of cultural heritage. Recurrent connections within multidimensional Long Short-Term Memory (MDLSTM) networks unveil a transformative potential, offering a digital refuge for historical documents (Skala, V., Singh, T.P., Choudhury, T., Tomar, R. and Bashar, M.A. eds., Md. Abul Bashar Editors, 2022).

Motivated by the surge in deep learning, the scientific landscape expands to embrace transformer architectures. Large Language Models (LLMs) and transformers, lauded for their prowess in natural language processing, beckon HTR into a new era of possibilities. This scientific trajectory seeks to bridge the chasm between traditional methodologies and the transformative capacities of transformers, redefining the theoretical boundaries within the dynamic field of HTR.

Amidst this scientific odyssey, the problem statement crystallizes around the intricacies of handwritten characters — variations in size, shape, and style pose formidable challenges. Traditional approaches like RNNs and CNNs, while demonstrating efficacy, grapple with the subtleties inherent in diverse handwriting styles.

The overarching scientific aim materializes — to augment handwritten text recognition by leveraging transformer models derived from large language models. The scientific objectives unfold meticulously: a comprehensive survey of existing HTR methods, the nuanced adaptation of transformer architecture to accommodate the intricacies of handwriting, meticulous data curation, and model training with a spotlight on practical applications, and a thorough scientific evaluation, dissecting robustness and generalization across diverse handwriting styles.

In this scientific narrative, HTR transcends its technological façade. It becomes a scientific guardian of cultural artifacts, a tool in the healthcare arsenal, and a testament to the evolving synergy between artificial intelligence and the intricacies of human

expression. This is the scientific overview of Handwritten Text Recognition — a landscape where algorithms grapple with the complexities of handwritten scripts, pushing the boundaries of scientific understanding and technological application.

2.6. Related Handwritten Text Recognition (HTR) Researches

The field of Handwritten Text Recognition (HTR) has witnessed notable advancements in recent years, spurred by the integration of deep learning techniques and the exploration of transformer-based models. However, several challenges persist, motivating ongoing research endeavors aimed at addressing key issues in HTR.

Historical documents pose a unique set of challenges for HTR due to their often degraded and diverse nature. Research by (Parres, D. and Paredes, R., 2023, August) identifies the transcription of historical documents as a particularly critical area within HTR. The existing dominance of convolutional recurrent neural networks (CRNN) with connectionist temporal classification loss in historical document recognition underscores the need for novel approaches. Despite the transformative impact of transformer models in natural language processing, their adoption in the context of historical document HTR remains limited.

TrOCR, a novel approach proposed by (Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z. and Wei, F., 2023, June.), addresses the persisting challenges in text recognition for document digitalization. Traditional methods relying on convolutional neural networks (CNNs) and recurrent neural networks (RNNs) for image understanding and text generation, respectively, often require additional language models for post-processing. TrOCR introduces an end-to-end solution with pre-trained image and text transformers, showcasing superior performance across printed, handwritten, and scene text recognition tasks. This indicates a promising avenue for achieving comprehensive text recognition without the need for additional post-processing steps.

The significance of transforming handwritten text images is further emphasized by (Bhunja, A.K., Khan, S., Cholakal, H., Anwer, R.M., Khan, F.S. and Shah, M., 2021), who propose Handwriting Transformers (HWT). HWT leverages a transformer-based network for styled handwritten text image generation, capturing both style-content entanglement and global/local style patterns. This innovative approach demonstrates the

potential to generate realistic styled handwritten text images, showcasing versatility in handling various writing styles and lengths of text.

Attention-based models, such as AttentionHTR proposed by (Kass, D. and Vats, E., 2022), tackle the challenge of limited training data in handwritten word recognition. By leveraging pre-trained models on scene text images and introducing a bidirectional LSTM-based sequence model with content-based attention, AttentionHTR proves effective on datasets like Imgur5K and IAM. This emphasizes the significance of transfer learning from synthetic scene text images to overcome data scarcity and enhance recognition performance.

Recognizing the need for effective word and character-level classification in handwritten documents (Balci, B., Saadati, D. and Shiferaw, D., 2017) and (Manchala, S.Y., Kinthali, J., Kotha, K., Kumar, J.J.K.S. and Jayalaxmi, J., 2020) focus on leveraging deep learning techniques with the IAM Handwriting Dataset. Both studies explore different architectures, including CNNs, LSTMs, and variants like VGG-19 and RESNET-18/34. These approaches demonstrate successful training for word-level classification, character segmentation, and achieving accuracy rates exceeding 90.3%.

The introduction of a Light Transformer-Based Architecture by (Barrere, K., Soullard, Y., Lemaitre, A. and Coüasnon, B., , 2022) addresses the limitations of traditional Transformer models in HTR, particularly when confronted with limited annotated data. This approach incorporates convolutional layers and Transformer components, maintaining reduced parameter counts suitable for small datasets. The hybrid loss function, combining Connectionist Temporal Classification (CTC) and Cross-Entropy (CE), contributes to training efficiency and improved performance, with the model achieving a Character Error Rate (CER) of 5.70% on the IAM test set.

HTR-Flor, presented by (de Sousa Neto, A.F., Bezerra, B.L.D., Toselli, A.H. and Lima, E.B., 2020), introduces a Gated Convolutional Neural Network with Bidirectional Gated Recurrent Units (Gated-CNN-BGRU) to address challenges in recognizing offline handwritten text. This model outperforms state-of-the-art CRNN approaches with fewer parameters, showcasing its efficiency in handling long sentences, variations, and noise in scanned documents. The success of HTR-Flor further emphasizes the potential for compact architectures in achieving superior recognition performance.

The study by (Coquenet, D., Chatelain, C. and Paquet, T., , 2020) introduces a Gated Fully Convolutional Network (GFCN) as a novel approach for unconstrained handwritten text recognition. By eliminating the need for Long Short-Term Memory (LSTM) cells and adopting convolutional layers with a gating mechanism, GFCN achieves competitive performance with fewer parameters. The incorporation of Depthwise Separable Convolutions (DSC) and a Fully Convolutional Network (FCN) further contributes to effective dependency modeling, showcasing the potential for innovative architectures in unconstrained HTR tasks.

In conclusion, while recent research has made significant strides in improving HTR through deep learning and transformer-based models, challenges remain in adapting these advancements to historical documents, addressing data scarcity, and optimizing model architectures for efficient recognition across various types of handwritten text. The collective insights from these studies lay the groundwork for continued exploration and innovation in the field of Handwritten Text Recognition.

2.7. Related HTR Researches Summary Table:

#	Study	Objective	Dataset	Tools & Techniques	Results	Opinion
1	Fine-Tuning Vision Encoder–Decoder Transformers for Handwriting Text Recognition on Historical Documents	Develop and assess a novel approach for Handwritten Text Recognition (HTR) on historical documents by fine-tuning pre-trained transformer models, specifically vision encoder–decoder models, with a focus on addressing challenges like	ICFHR 2014 Bentham dataset, ICFHR 2016 Ratsprotokolle dataset, Saint Gall dataset	Fine-tuning pre-trained transformer, particularly vision encoder–decoder models.	ICFHR 2014 Bentham dataset: 6.9% WER, ICFHR 2016 Ratsprotokolle dataset: 14.5% WER, Saint Gall dataset: 17.3% WER	Does not compare the proposed approach to existing fine-tuning methods for vision

		limited training data.				transformers.
2	TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models	The objective of this paper is to propose TrOCR, an end-to-end Transformer-based OCR model for text recognition with pre-trained CV and NLP models.	5,427 handwritten fonts to synthesize handwritten textline images by the TRDG2, an open-source text recognition data generator.	CRNN CNNbase d encoder and RNN-based decoder	The pre-trained models on the vision task improve the performance of text recognition models, and the pure Transformer models are better than the CRNN models and the Tesseract on this task	Lacks comparison with other Transformer-based OCR models.
3	Handwriting Transformers	Introduce a novel transformer-based approach, HWT, for styled handwritten text image generation. Aim to learn style-content entanglement, capture global and local style patterns through self-attention mechanisms, and achieve realistic results in handling arbitrary text lengths and diverse	-	Transformer-based network (HWT) with an encoder-decoder attention mechanism	HWT generates realistic styled handwritten text images	Limited comparison with existing methods for styled text generation.

		writing styles.				
4	AttentionHTR : handwritten text recognition based on attention encoder-decoder networks.	Develop an attention-based sequence-to-sequence model for handwritten word recognition, addressing data scarcity through transfer learning from pre-trained scene text images. Evaluate the proposed model on multi-writer datasets (IAM and Imgur5K) and conduct in-depth analyses of error cases.	IAM Dataset, Imgur5K Dataset	ResNet feature extraction and bidirectional LSTM-based sequence modeling stages forming an encoder.	Word Error Rate (WER): Achieved competitive results across different transfer learning strategies. IAM Dataset WER: 15.40% (word level) IAM Dataset CER: 6.50% (character level)	Does not investigate the impact of different pre-trained models on the performance.
5	Handwritten Text Recognition using Deep Learning	The research develops a system using CNNs and LSTMs to classify handwritten words, aiming to translate handwritten text into digital format for user-friendly document conversion.	IAM Handwriting Dataset, with 1500+ forms and 600+ writers, is the main resource, providing a diverse and verified dataset for model training.	Padding, rotation, VGG-19, RESNET-18/34 enhance word-level classification robustness with ADAM optimizer.	Word-level models (VGG-19, RESNET-18/34) succeed with large softmax challenges. Character-level classification enhances word recognition, promising efficient handwritten document conversion.	Does not explore the use of more recent and potentially more efficient architectures, such as Transformers.
6	Handwritten Text	Develop a Handwritten Text	IAM Handwriting	Utilized CNNs	Achieved over 90.3% accuracy,	Limited compari

	Recognition using Deep Learning with TensorFlow	Recognition (HTR) system for accurate word classification in digital conversion.	Dataset: 1500+ forms, 600+ writers, diverse and verified, with detailed transcription and coordinates.	(VGG-19, RESNET-18/34) for word-level classification, LSTMs for character segmentation, and Tesseract model for preprocessing.	successfully addressing challenges in large softmax layer output and offering a robust solution for handwritten text to digital conversion.	son with other HTR systems.
7	A Light Transformer-Based Architecture for Handwritten Text Recognition	Develop a lightweight Transformer-based architecture for Handwritten Text Recognition (HTR) with a focus on small datasets, aiming to achieve state-of-the-art results without the need for extensive training data.	The research employs the IAM Handwriting Dataset. With 400 text-line images samples training and evaluation.	Proposes Light Transformer architecture, integrating CNNs and Transformers. Employs hybrid loss (CTC + CE) for training.	Light Transformer achieves 5.70% CER on IAM test set without additional data. With synthetic data, CER improves to 4.76%, outperforming a larger variant (Large Transformer). Hybrid loss proves crucial for efficient training.	Does not provide a qualitative analysis of the generated outputs.
8	HTR-Flor: A	Develop a Gated-	The research	PyTorch	The Gated-CRNN	Limited

	Deep Learning System for Offline Handwritten Text Recognition	CRNN architecture to optimize Handwritten Text Recognition, achieving higher accuracy and efficiency with reduced parameters, particularly suitable for offline scenarios.	utilized the IAM Handwriting Database, a diverse collection of handwritten texts.	and TensorFlow for model development,. Gated-CRNN architecture, blending CNN and RNN	achieved state-of-the-art results on IAM dataset, outperforming existing methods. Notably, it demonstrated superior accuracy and reduced parameter count, making it suitable for resource-constrained environments.	explanation of the design choices for the Gated-CRNN architecture.
9	Recurrence-free unconstrained handwritten text recognition using gated fully convolutional network	The research aims to enhance unconstrained handwritten text recognition by introducing a Gated Fully Convolutional Network (GFCN), providing an efficient alternative to LSTM-based models, overcoming issues like excessive parameters and sequential processing during training and prediction.	Utilizing RIMES and IAM datasets, the research employs widely-accepted benchmarks for handwritten text recognition. Images are preprocessed by resizing to 64 pixels in height	Implemented in PyTorch, the GFCN model incorporates CTC loss, Depthwise Separable Convolutions (DSC), and various regularization techniques	The Gated Fully Convolutional Network (GFCN) excelled in unconstrained handwritten text recognition, achieving competitive CERs of 4.35% on RIMES and 7.99% on IAM datasets. With a low parameter count, GFCN displayed efficiency in training and	Lacks a detailed analysis of the learned features and filters in the GFCN, limiting the understanding of its decision-making process

			while retaining original width, forming the basis for training, validation, and testing.	. Training leverages the Adam optimizer on a GPU (V100 16GB), evaluating performance with CER and WER metrics.	prediction times, affirming its potential for text recognition tasks.	and identifying potential biases.
--	--	--	--	--	---	-----------------------------------

2.8. Summary

Chapter II provides an in-depth exploration of advancements in Handwritten Text Recognition (HTR), driven by the integration of deep learning methodologies. The chapter introduces the evolution from Convolutional Neural Networks (CNNs) to Recurrent Neural Networks (RNNs) and the recent surge in transformer-based approaches. The narrative delves into challenges faced in recognizing diverse handwriting styles and highlights the transformative potential of large language models and transformers. The chapter discusses the problem background, emphasizing the limitations of traditional methods and the need for a new protagonist – transformer models. It outlines objectives such as a comprehensive literature review, adaptation of transformer architecture, diverse data curation, and model training. The storyline evolves with a focus on image processing, Optical Character Recognition (OCR), and the broader context of handwritten text recognition. The scientific overview emphasizes HTR's role in preserving cultural artifacts and healthcare applications. The chapter concludes with a review of related research, showcasing innovative approaches, challenges in historical document recognition, and the potential of transformer architectures in HTR.

Chapter Three

Methodology

3.CHAPTER III

3.1. Introduction to the Methodology

In our efforts to make the machines capable of reading the handwriting as we humans do naturally, this chapter is a compass that outlines each of the stages of research work. While human beings can effortlessly assimilate and reproduce recognition of handwriting, carrying out this function in machines has to be done in a procedural and scientific manner. The methodology that is presented in this regard does not remain as a theory but addresses practical issues on the subject by showing how machines can be taught to read and comprehend script and cursive handwriting.

This study investigates the core of Transfer Learning, which is an important concept that can be likened to mentorship in the process of learning. Our model does the same and, as an apprentice, makes use of the knowledge contained in the pre-trained language models. Such models do not come clean as they have several practices embedded within them, already and open to the contextual understanding and application of language. The basis of this is **fine-tuning** where the aim is to enable the model to adjutant and modify the think therefore i am existence, what is drawn from the wires of its component parts and its construction. This is so because it would be like how a student learns the basics of mathematics and is then able to solve even harder and much complex worded problems. What is desired is to expedite the learning of the model so that it can be able to recognize different types of writing effortlessly and accurately in the shortest time possible.

With this groundwork, the stage is further conducted—the evolution of a model over a training phase. Training of a model is often compared to adjusting a musical instrument, as it is aggravating and call for a rational approach. This fine-tuning involves much more than adjusting hyper parameters and picking optimization strategies – it includes, for instance, employing data augmentation, among other techniques. Doing so with each iteration fine-tunes the model, hence bringing it closer to a more perfect Everton equilibrium where it is able to generalize with high accuracy without losing fit. This is a crucial step in developing a model that

is able to read various handwriting styles and still maintain the same level of efficiency.

The last process is evaluation stage- a very significant single period in which we see if that is all of the capabilities of a model. During this period the model is tested on all the metrics as well as the benchmarks i.e. Character Error Rate (CER), accuracy, precision etc. That allows that stage for providing some qualitative assessment of the model and its abilities and full understanding what exactly should be improved in that model. Even though this stage provides answers to the questions already formulated, it also raises additional ones, which means that the path of development and improvement has yet to come to an end.

This approach elucidates a progression, not simply of steps or actions but of questions and investigations of a scientific nature. Each and every step including but not limited to the query expansion or decision on datasets for a particular objective is taken rather systematically and rationally owing to the problems and advantages allied with transforming natural language to a computer's style of writing. It is this path that we have adopted which is rooted in appreciating the difficulty of the work and the desire of making the results better which keeps us on the way to developing a robust and efficient handwritten text recognition system.

This complex undertaking encompasses much more than merely choosing appropriate implement such as transformer based architectures or large language models LLMs. Models like GPT-3, BERT, and others have revolutionized the way machines understand and generate human language. Such models are able not only to recognize patterns in data but also to understand what those patterns of latent variables contextually infer. They can read even the most difficult slants and curves of a person's hand writing and interpret the motives behind those handwriting strokes convincingly.

In deploying large language models for this task, the process entails much more than learning to read letter by letter or word in some cases. These models are already trained on a lot of language knowledge and therefore can appreciate text as a whole and thus better predict meaning and transcription accuracy. When the model is tasked with recognizing handwritten documents, it is not a matter of

starting from a completely blank slate but rather enhancing that of the existing knowledge, which allows better adaptation to the different handwriting styles. A person who has read widely can easily understand smartly different regional accents, and so our model learns to make sense of many different styles or variations of writing.

Returning back to LLMs' broad capabilities and honing in on the handwriting aspect we will engage in, it is evident that this combination is not merely tactical but rather intricate interweaving of strategies. It is an integration of transformer characteristics and LLMs in a very purposeful manner drawing on art and science to achieve a certain expression. It is through this approach that the designs of reading machines which understand the aesthetic aspect of written text are developed, thereby, taking a leap into another dimension where expressions of thoughts and feelings through art meets technology.

In this sense, we strive to improve on existing systems of handwritten text recognition to a level that has never been reached before by combining innovative models with existing anthropocentric knowledge. This chapter thus has a methodological purpose, which is to establish the basis for our practice-oriented research and explain how theoretical ideas transform into practical ones, in our case, owing to the intention to build machines able to read with human comprehension.

3.2. Data Collection and Preprocessing

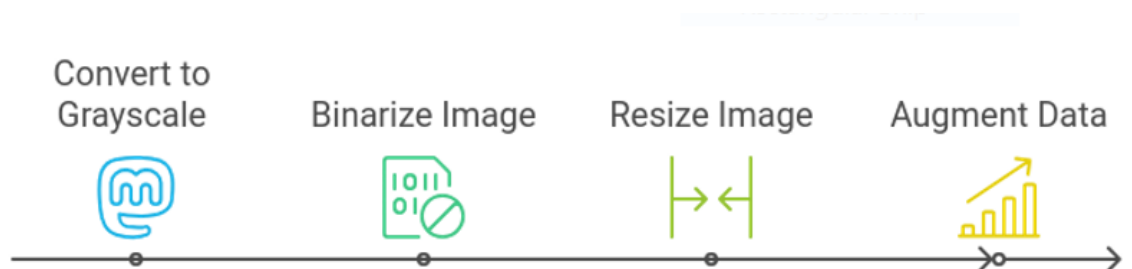
3.2.1. Data Collection

The dataset selected for this research is the **IAM Handwriting Dataset**, a widely used benchmark for handwritten text recognition tasks. It consists of a diverse range of handwritten samples, including various writing styles, sizes, and formats. This dataset allows the model to generalize better and tackle the inherent variability of human handwriting.

3.2.2. Preprocessing

In a bid to improve the quality of input data, these preprocessing steps were implemented:

- Normalization: The images were converted to grayscale in order to decrease the computation burden especially since color has no significance in handwritten text detection.
- Binarization: Text background separation was achieved using the follow thresholding technique. The thresholding technique also known as binary image processing helps to clearer transform images by eliminating severe object edges.
- Resizing: The resized images were used for the transformer based model at all times to ensure every input dimension was uniform.
- Data Augmentation: In terms of the various methods it was rotation, scaling, and the addition of noise that helped to address the issue of the different styles of handwriting in order to ensure better model generalization.



These preprocessing steps made it possible to utilize the dataset in transformer-based architecture focusing on consistency and enhancement of the model efficacy.

3.3. Model Selection and Architecture

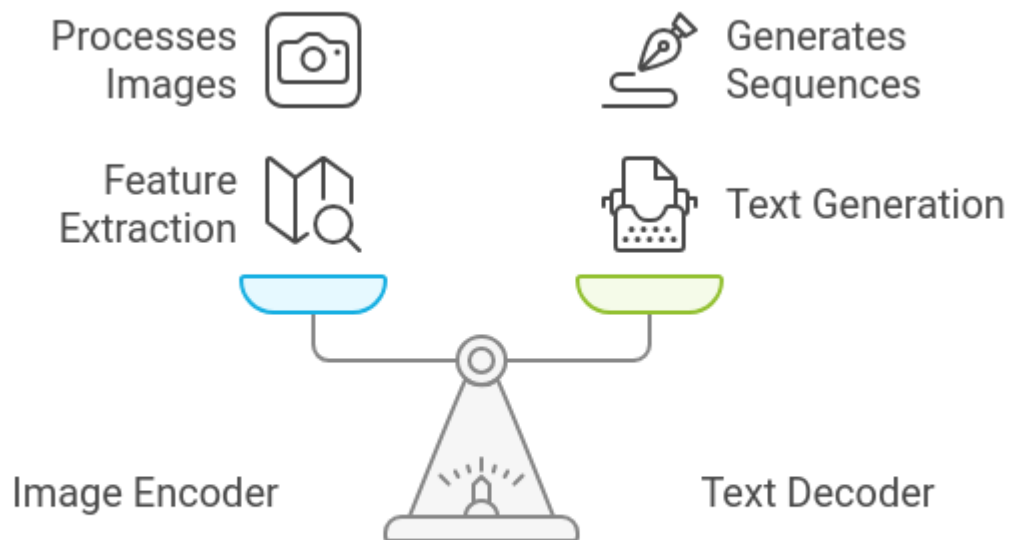
3.3.1. Transformer-based Model

The TrOCR (Transformer OCR) model is the core mechanism of the research, which contains an image encoder and a text decoder.

The model is effective for recognizing handwritten texts because it visualizes the features and produces text sequences.

- Image Encoder: The encoder deals with the preprocessed images of the handwriting and pulls out features of interest.
- Text Decoder: The decoder converts the visual features of the image into text and converts them into a sequence using the attention mechanism of the transformer architecture to map complex handwritten characters.

Furthermore, the model was adjusted for the Handwritten text recognition tasks based on fine-tuning of Bullinger-general model which is a general one but trained on historical documents available in the archives already.



Roles and functions of TrOCR components.

3.4. Transfer Learning and Fine-tuning

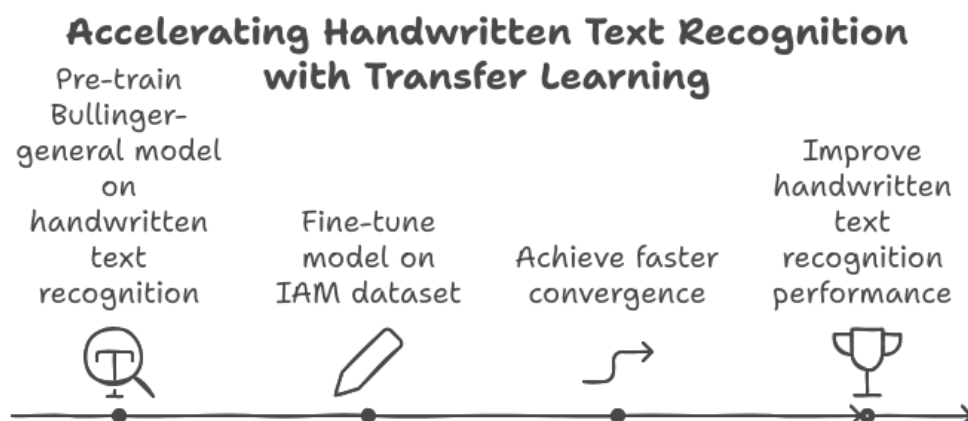
3.4.1. Transfer Learning Approach

Model, authors were able to recognize handwritten text but also develop compounding skills which integrated with the primary task. As a result, this pre-training eased the struggles of learning complex text written in different styles when incorporating the model with the IAM Handwriting Dataset, since less retraining was needed.

Equally, just as a person who understands a certain language may learn a certain other dialect of that language in no time, the pre-trained model already knew about the organization of language fabric and text even before being introduced to the task. This enabled good generalization when encountering the various types of handwriting present in the IAM dataset. Thanks to the previous experience of the model, the process of fine-tuning was expedited; therefore the model required a shorter number of iterations to master the peculiarities of the new task.

In this case, the adjustments were made to the weights and parameters of the pre-trained model in order to enhance its property of tolerating the variability and complexity of handwriting recognition task. This is similar to how an artist perfects one's skills for the purposes of a more focused task. With the advantage of the Bullinger-general model, authors could not only solve the problem of handwritten texts recognition. However, to the main activity were integrated developing skills added to the primary task. Thus, this pre-training alleviated the difficulties in learning a highly complex dressed text, when deploying the model with the IAM Handwriting Dataset, as more retraining was avoided.

S

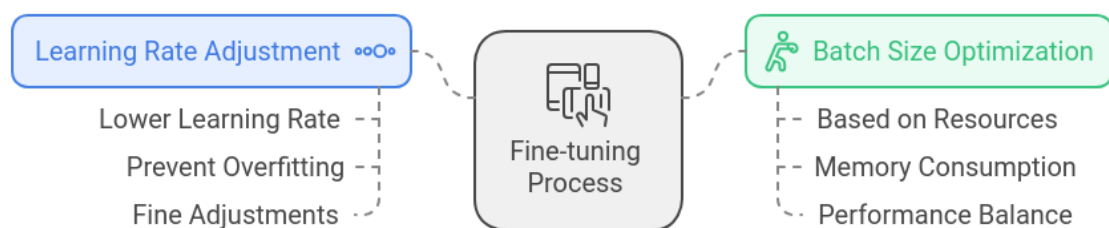


3.4.2. Fine-tuning Process

The model was further improved by utilizing the IAM Handwriting Dataset, which enhanced its ability to face the challenges within this dataset. Model tweaking is critical for handwritten texts because they vary a lot in styles and accuracy and generalization must be guaranteed. The effective technique of fine-tuning involved the following significant changes:

- **Adjustment of the Learning Rate:** A lower learning rate was used to avoid overfitting and also to restrict the changes to the weights of the pre-trained model increases. This resulted in effective learning during fine-tuning where the model was trained on the new data without losing the essential information learned in the previous training. This means that learning was consciously deteriorated, this helped especially where there were complex writing styles that were extremely different from one person to another.
- **Tuning of the Batch Size:** The batch size was modified to avoid a case where performance exceeds efficiency or vice versa. As larger batch sizes tend to be costly in terms of memory, the batch size was optimized as per the processing capabilities existent, lest put strain on the net with a learning hitch. This also contributed to the improvement of the training process in that the model was able to improve gradually without causing stress to the resources allocated.

These fine-tuning modifications, intended exclusively for the IAM dataset, altered the model in such a way that it understands various handwriting styles, without losing the essence of knowledge acquired from previous tasks. Through learning rate and batch size improvement, the model perfects its predictions, achieving accuracy and flexibility at the same time.

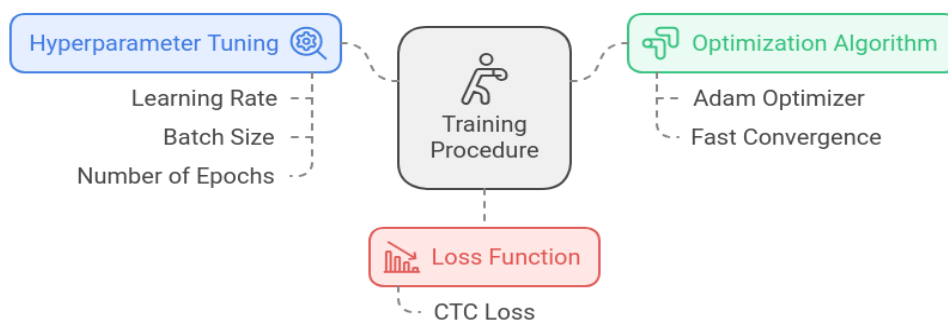


3.5. Training and Optimization

3.5.1. Training Procedure

The experiment has been administered by following a specific sequence that ensured both model capability and task relevance to handwritten text recognition. Every aspect of the experiment was planned thoroughly in order to achieve high accuracy and high generalization of the model provided with different handwritten inputs. The main steps of the training process are presented in detail under:

- **Hyperparameter Optimization:** Optimization of key hyperparameter such as learning rate, batch size, epochs had to be carried out in order to get the best performance of the model. A lot of experiments were done to derive these values so that the model learns effectively without overfitting. The value of learning rate was set in a way to ensure progressive changes of the model weights, sufficiently stepped up. The batch size was also set in such a manner so that there wouldn't be excess memory load with the speed of processing being optimum. The epochs were subject to early stopping criteria where an epoch had to be sufficient for the model to be able to generalize well hence trained to the extent of overfitting on the training set. All these parameters were significant in striking the right balance between accuracy and the ability to generalize.
- **Loss Function:** Since handwritten text recognition is essentially an output-input sequence, not every input of handwritten character has its corresponding aligned output of recognized characters. Hence in the recognition task the CTC loss function was considered. This is useful for this task since alignment of input images with the resulting text is not necessary. Rather, it allows the model to learn how to find the order of character's without the need of sticking to the spaces or if some characters overlap each other in the same handwriting. This loss function is very important for the model to draw constructively to the problem at hand.

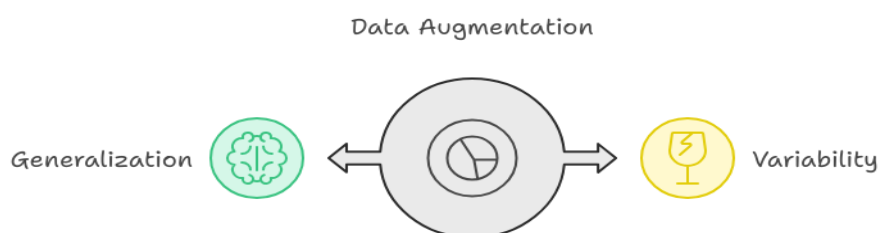


3.5.2. Data Augmentation During Training

During the implementation of the strategy, data augmentation was firmly maintained as a means of improving and diversifying the content inside the dataset to recreate various writing conditions. This process was designed to replicate real life scenarios of writing variations including slant, height of the word or letter, length of spaces between letter combinations among many others in order to ensure that the end user models through training interacts with a wide range of styles. By means of augmentation of the data, we in fact were able to increase the size of the training set in general without additional samples which had a positive impact on model generalization. Techniques of data augmentation such as rotation, scaling, translation, and adding noise were applied to imitate the normal variations of handwritings. These transformations, in turn, enabled the model to train on a realistic level of distortions in letters, for instance, slant-wise distortions, thickness levels among the strokes and many others. Since such evidence was presented to the model each time during training, it was able to sustain a fair level of tolerance of how all forms of writing, although different, could still be written, hence, readying it for the expectations of application in real-life scenarios where external elements are beyond control.

This in turn can be considered as an individual introducing oneself into the art of reading—having before only manuscript pages—from the most elegant penmanship to the most scrawled alterations of hurried letters and back. This aspect of training is extremely important, increasing the versatility of the model, such that it does not only target known handwriting styles but is able to target lots of unknown styles as well without compromising on accuracy.

In this way, data augmentation enhanced the model's ability to generalize quite effectively. This strategy was very important in helping the model cope with the differences in individual handwriting patterns so that it could still work well with a wide range of writing samples. The end product is an improved recognition system that is flexible and efficient.



3.6. Model Evaluation and Metrics

3.6.1. Evaluation Metrics

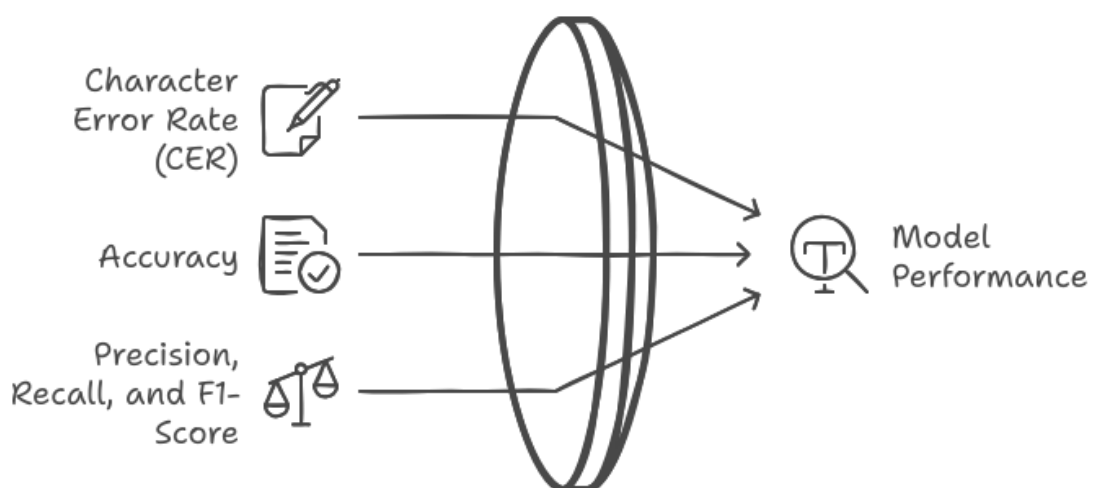
The evaluation of the model's efficacy in handwritten text recognition was based on a series of established and advanced performance measures to aid in providing a full assessment. The metrics employed were purposefully selected to measure various parameters of the transcription accuracy of the model and the transcription accuracy of the model across different handwriting styles.

- ❖ **Character Error Rate (CER):** As one of the main measures used to assess performances in handwriting recognition systems, CER is computed as the number of character-level mistakes, which include insertions, deletions, or substitutions, against the number of characters in the ground truth. This metric reveals the capability of the trained model of writing by hand copying as closely as possible, without respect to attention to the content. A lower CER suggests that the model does not make a lot of errors highlighting its ability even to understand complicated or erratic writing styles. CER is very useful in handwritten text recognition because it gauges the model's response to character changes, including variation in writing styles or small curvature in different letters.
- ❖ **Accuracy:** This measurement indicates the ratio of the correctly identified sequence of the text to the respective ground truth limits it has to conform. Accuracy provides a wide picture of the model capability to guess the correct output transcription by considering many factors within one string transcribed and even if that string consisted of several words and characters and whole text within those rather than in individual character errors which CER considers. While Character Error Rate considers the error rate is at the level of each character, accuracy is at the level of a generalized transcription across rows or sequences of text. Where high accuracy translates to the ability of the model to recognize a given content without alteration in the recognition model.
- ❖ **Precision, Recall and F1-Score:** These metrics provide further quantification of the model evaluation by looking at how many characters are falsely recognized and how many are not recognized.

- ✓ **Precision** is the ratio of the correctly predicted characters to the total number of characters predicted as correct by the model, thus giving an idea about the specificity of the model.
- ✓ **Recall** is concerned with the ability of the model such that all the relevant resulting characters from the handwriting are included, that is, its ability to contain all the writing.
- ✓ **The F1-Score** is simply the Coefficient of the Two Precision and Recall and serves to provide a measure in which the number of false positives and false negatives is factored in. High level of the F1-Score means that the model is able to balance precision and recall effectively allowing for accuracy and coverage in the transcriptions.

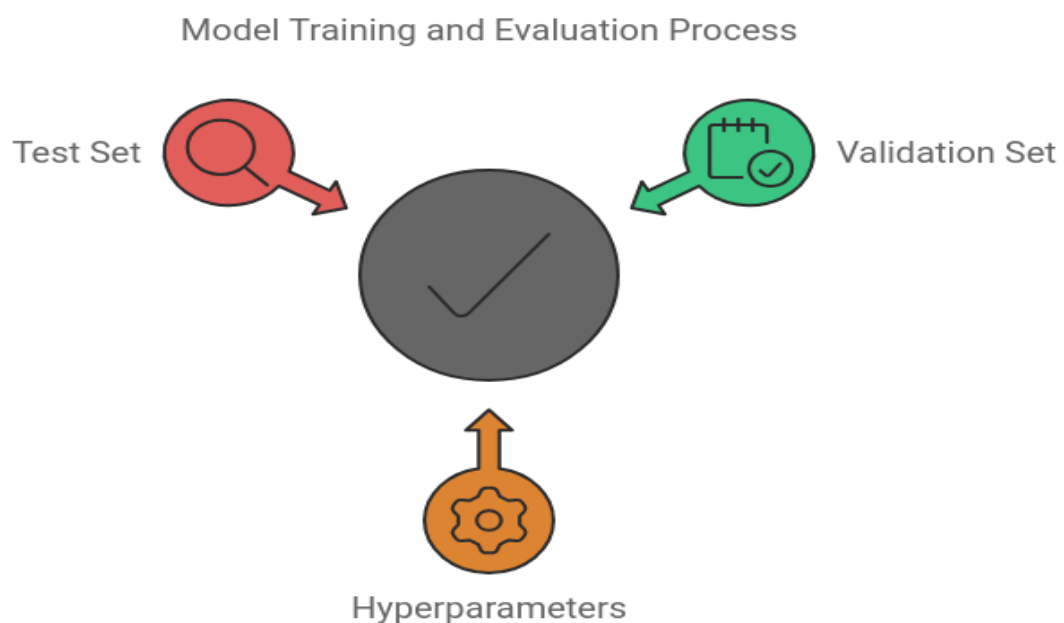
These metrics, when viewed as a whole, create a substantial analytical embedded system that addresses all dimensions of the model's output. Observing how the model performs on character error rate (CER) which measures the accuracy of individual characters, the overall accuracy of the transcriptions (content accuracy rate) and the ratio of the number of correct positive results retrieved to the total number of retrieved positive results (as well as false negatives and F1-Score) gives an understanding of how the model performs for handwritten text recognition. This exhaustive assessment helps in concluding that the developed model is not just accurate but can perform well under various kinds of handwriting.

Handwritten Text Recognition Model Evaluation



3.6.2. Validation and Testing

Throughout the course of training, a distinct validation set was used to periodically evaluate the performance of the model, to inform optimization of hyperparameters. This validation set, which was not part of the training data, proved important in controlling overfitting by showing, in real time, how well the model performs on data not present in the training set. With this in mind, we were able to improve or reduce the learning rate, batch size and some other parameters as necessary to ensure that the system would not only perform to its best accuracy but also do so within reasonably minimal time limits. The validation set also helped us monitor the development of significant performance indicators as Character Error Rate (CER) (c) and accuracy when the model was training. This provided very useful evidence that triggered several changes in the training process, making it possible to enhance the performance of the model up to a given point without tailoring it too much to the training set. After all the training is accomplished, the model undergoes assessment using a held out test set which is simply a sample that was not used for training and validation processes. This sample served the utmost purpose in evaluation of the model's generalization ability which is its ability to perform well on new and unseen data in recognizing handwritten texts. Judging from this held-out set provided an evidence of the actual performance of the model in practical applications in that the system did not simply learn the training set curves but was able to learn the handwriting within a wide range of variations.



The concluding model was assured of being both precise and resilient by verifying the model at every stage of the training and then retesting it again. It was able to generalize on multiple forms of handwriting samples and still remain accurate. The two-stage evaluation that was carried out in a model, that is, validation and testing, was critical in developing a model suitable for use in real-life scenarios.

3.6.3. Comparison with Baseline Models

At every step, the suggested transformer-based model was taken through a series of rigorous experimental validation against some of the standard practices, which also included the conventional Convolutional Neural Networks (CNNs) and Recurrent Neural Network (RNN) techniques as they are normally known to be applied during handwritten text recognition. This was necessary in ascertaining whether the transformer-based framework, which incorporates pretrained language models with attention techniques, will be better than these existing ones in terms of performance metrics.

The outcome of this experiment was quite simple and straightforward: the model based on the transformer architecture was more accurate in handwriting recognition across varying handwriting styles in comparison to other models, namely CNNs and RNNs. Whereas CNNs are proficient at analyzing images by extracting some spatial features and RNNs cope well with time-dependent sequences, both these systems find it difficult ultimately with the variability and sophistication of writing, especially when considering different handwriting styles with uneven spaces between letters.

In contrast, the transformer's use of self-attention enabled the model to incorporate long-range context into the representation, which allowed the model to pay attention to every style of handwriting present. The attention-enabled model presented the ability to attend to specific regions of interest within the input, allowing for alterations in the size, thickness, and even the font of the characters. Such ability was not found in typical CNNs and RNNs.

Furthermore, another notable benefit was the effectiveness of the model based on the transformers architecture. The model, because of the processing capabilities of transformers, was less efficient than RNNs, which process sequences in order and consequently are more time and resource-consuming, capable of processing copious datasets more quickly than RNNs. Besides, the employed architecture of the transformer

led to a more rapid training process encouraging faster convergence as well which in turn contributed to less computational resources being used while providing enhanced performance.

On the below figure 3.9 the performance metrics of the transformer model and baseline CNN and RNN models are illustrated and the superiority of the transformer in terms of both accuracy and efficiency is evident.



Overall, the analysis showed that the transformer-based model not only outperforms in varying handwritings recognition with an increased precision but also does this faster than the conventional methods. It explains the transformer architecture as an enhancement over previous techniques, thus making it suitable for the challenging task of handwritten text recognition.

3.7. Conclusion

This paper's approach combines state-of-the-art transformer architectures, large language models' (LLMs) transfer learning, and efficient data processing techniques to enhance the task of handwritten text recognition (HTR). The strategy in each of the components tends to enhance the understanding and transcription of various forms of handwriting by the model with speed and accuracy.

The model utilizes the power of transfer learning to augment its capabilities using available language models, and it this enables the model to be copious in use and handle the hurdles of written text successfully. This is similar to how people tend to take what they have learned and apply it in new situations, therefore, the model can visualize how the letters are structured and the handwriting styles as well as their differences in a much better way.

Effective data preprocessing strategies, such as image binarization, resizing, and augmentation are put in place so that the model has experienced varying handwriting conditions to improve its generalization performance to different handwriting styles. These preprocessing activities are very important because they help in preparing the hypotheses for the active consideration of the transformer model. Hence, the model is able to view the relevant aspects of the handwritten text without distractions caused by noise in the original pictures.

The performance of the model is enhanced step by step, by means of strict range of training and tuning procedures, to the level of accuracy and efficiency that is beyond that of other approaches such as CNNs and RNNs. The attention mechanisms of the transformer allow the model to get used to the variations brought about by different handwritings within the model, thus making it applicable to various forms of writing.

The findings of this study bring forward the evidence that HTR significantly benefits from the use of transformer-based architectures due to the effectiveness of the model compared with the prior models in both accuracy and the model complexity. This approach does not only bring progress in the contemporary modes of handwritten text recognition but also presents opportunities for possibilities in the projected modes within the sphere transforming the application of transformers and LLMs in HTR tasks.

Chapter Four

Implementation

4. CHAPTER III

4.1. System Overview

The implementation delivers an English handwritten text recognition (HTR) research and application pipeline. The system has two integrated layers: (1) a reproducible batch evaluation pipeline for measuring Character Error Rate (CER) and Word Error Rate (WER) on the IAM Handwriting Database, and (2) a Gradio web application for interactive line transcription. Both layers share the same core inference and metrics modules under `src/htr/`. The pretrained checkpoint `microsoft/trocr-base-handwritten` is used throughout.

Technology choices were guided by three principles: reproducibility (every result traces to a config file and an output artifact), interoperability (standard open-source ML libraries rather than custom frameworks), and separation of concerns (data preparation, model inference, metric computation, and user interface are implemented as distinct modules that can be tested and replaced independently).

4.2. Repository Structure

The project is organized as follows:

- `configs/`: YAML experiment definitions (`iam_trocr_handwritten`, `iam_demo`, `iam_tesseract`)
- `src/htr/`: Shared library — models, inference, metrics, and batch evaluation
- `scripts/`: Data preparation, evaluation CLI, figure generation, and experiment runners
- `app/gradio_app.py`: Gradio web demo for English IAM line images
- `data/iam/`: IAM dataset exports (processed splits and demo examples)
- `experiments/`: Evaluation outputs (`metrics.json`, `predictions.csv`, figures)
- `pyproject.toml` / `environment.yml`: Dependency and environment specifications

4.3. Technology Stack — Detailed Overview

This section explains each technology used in the implementation, its purpose within the system, and the rationale for choosing it over alternatives. Dependencies are declared in `pyproject.toml` (`pip install -e .`) and `environment.yml` (conda alternative).

4.3.1 Python 3.9+ (recommended 3.10)

Purpose: Serves as the sole implementation language for data scripts, the shared HTR library, batch evaluation, and the Gradio web application.

Reason for selection: Python is the de facto standard for machine learning research and prototyping. Its ecosystem provides first-class support for PyTorch, Hugging Face Transformers, and Gradio — all required by this project. Version 3.9+ is mandated by `pyproject.toml`; Python 3.10 is specified in `environment.yml` for conda users because it balances stability with broad library compatibility.

Usage in this project: All modules under `src/htr/`, `scripts/`, and `app/` are Python. The package is installed editable via `pip install -e .` so imports resolve consistently across scripts and the app.

4.3.2 PyTorch (torch) and torchvision

Purpose: Provides the deep learning runtime: tensor operations, GPU acceleration, model loading, and autoregressive text generation during TrOCR inference.

Reason for selection: TrOCR and Hugging Face Transformers are built on PyTorch. PyTorch offers dynamic computation graphs, mature CUDA support, and direct integration with `VisionEncoderDecoderModel`. `torchvision` complements `torch` for image tensor handling. Alternatives such as TensorFlow or JAX were not chosen because the TrOCR checkpoint and processor are distributed through the Hugging Face PyTorch ecosystem.

Usage in this project: `models.py` loads weights with `VisionEncoderDecoderModel`.

`from_pretrained()`, moves the model to cuda or cpu via `model.to(device)`, and runs `model.generate()` inside `torch.no_grad()` during inference.

`evaluate.py` uses `torch.utils.data.DataLoader` for batched evaluation.

4.3.3 Hugging Face Transformers

Purpose: Supplies the TrOCR model architecture (VisionEncoderDecoderModel), the matched TrOCRProcessor (image preprocessing and text tokenization/decoding), and the standard from_pretrained() API for loading microsoft/trocr-base-handwritten.

Reason for selection: Implementing TrOCR from scratch would require replicating the ViT encoder, BART decoder, and tokenizer pipeline — hundreds of lines of error-prone code. Transformers provides battle-tested, paper-faithful implementations maintained by Microsoft and the open-source community. It also ensures that processor and model weights stay paired, which is critical for correct preprocessing and decoding.

Usage in this project: TrOCRProcessor.from_pretrained(checkpoint) loads both image_processor and text tokenizer.

VisionEncoderDecoderModel.from_pretrained (checkpoint) loads encoder-decoder weights. Both are called in src/htr/models.py inside load_model_bundle().

4.3.4 Hugging Face Hub (huggingface_hub)

Purpose: Handles automatic download, caching, and versioning of the TrOCR checkpoint and processor files from the Hugging Face model registry on first use.

Reason for selection: Storing large model weights (hundreds of megabytes) inside the Git repository is impractical. The Hub provides a reliable CDN-backed distribution channel with local disk caching, so the same checkpoint is fetched once and reused across evaluation runs and the Gradio app. sentencepiece (a Hub dependency for BART tokenization) is pulled transitively.

Usage in this project: When load_model_bundle() or from_pretrained() is first called, weights are downloaded to the local Hugging Face cache (~/.cache/huggingface/). Subsequent runs load from disk.

4.3.5 TrOCR Architecture (ViT Encoder + BART Decoder)

Purpose: Performs the core handwritten text recognition task: encoding a line image into visual features and autoregressively decoding them into a text transcription.

Reason for selection: TrOCR (Li et al., 2021) is the state-of-the-art transformer approach for OCR/HTR and the focus of this thesis. The Vision Transformer (ViT) encoder captures spatial relationships in handwriting better than classical CNN feature extractors for this task. The BART decoder leverages pretrained language modelling to produce

coherent word sequences. The checkpoint microsoft/trocr-base-handwritten is pretrained specifically on English IAM handwriting, making it the correct model for this benchmark without additional fine-tuning.

Usage in this project: The encoder processes RGB line images into pixel tensors. The decoder generates token IDs up to `max_new_tokens=128`, which are batch-decoded into strings in `ModelBundle.generate_batch()`.

4.3.6 Gradio (4.x)

Purpose: Provides the web-based user interface for the interactive HTR demo: image upload, transcription display, optional ground-truth CER feedback, and an examples gallery.

Reason for selection: Building a custom HTML/JavaScript frontend would add significant development overhead without improving research outcomes. Gradio is designed for ML demos: it handles file upload, image preview, event callbacks, and local/server deployment in a few lines of Python. It also supports Blocks API for structured layouts and integrates natively with PIL images returned by the inference pipeline.

Usage in this project: `app/gradio_app.py` defines a `gr.Blocks` interface with `gr.Image`, `gr.Textbox`, `gr.Button`, and `gr.Examples` components. The `btn_submit.click` handler calls `process_image()`, which delegates to `recognize()` and `compute_cer()` from `src/htr/`.

4.3.7 Pillow (PIL)

Purpose: Loads, converts, and passes image data between the filesystem, Gradio upload widget, and the TrOCR image processor as RGB PIL Image objects.

Reason for selection: Pillow is the standard Python imaging library and the format expected by both Gradio (`type='pil'`) and Hugging Face image processors. It handles PNG/JPEG decoding, colour space conversion (`.convert('RGB')`), and saving during the IAM data export pipeline. OpenCV was not used because TrOCR preprocessing is handled entirely by TrOCRProcessor and no custom computer-vision operations (binarization, deskewing) are applied in this implementation.

Usage in this project: `infer.py` converts all inputs to RGB via `_load_image()`. `prepare_iam_lines.py` saves IAM line images as PNG files. `evaluate.py` opens image paths with `Image.open()` before batching.

4.3.8 jiwer

Purpose: Computes Character Error Rate (CER) and Word Error Rate (WER) using Levenshtein edit distance, aligned with standard HTR evaluation practice.

Reason for selection: CER and WER are the primary metrics defined in docs/RESEARCH_SCOPE.md. jiwer is a lightweight, well-tested library dedicated to speech and text recognition metrics. It was chosen over manual edit-distance implementations (error-prone) and over the Hugging Face evaluate library's CER metric because jiwer gives direct control over normalization and integrates cleanly with per-line and corpus-level aggregation. The evaluate package is listed in dependencies for compatibility but jiwer is the active metric backend.

Usage in this project: src/htr/metrics.py calls jiwer.cer() and jiwer.wer() after case-sensitive whitespace normalization. summarize_errors() uses jiwer.process_words() for alignment-based error pattern analysis. The Gradio app calls compute_cer() for live per-image feedback.

4.3.9 Hugging Face datasets

Purpose: Downloads and loads the Teklia/IAM-line dataset with official train, validation, and test splits directly from the Hugging Face Hub.

Reason for selection: The IAM Handwriting Database is the thesis benchmark. The Teklia/IAM-line distribution on Hugging Face provides clean, pre-split line-level data with image and text columns, avoiding manual parsing of the original IAM XML/ASCII distribution. Using datasets ensures reproducible splits (6,482 / 976 / 2,915 lines) that match published benchmarks.

Usage in this project: scripts/download_iam.py calls load_dataset('Teklia/IAM-line') and exports each split to Parquet. scripts/prepare_iam_lines.py reloads Parquet (or streams from Hub) and writes per-split image folders and labels.csv files.

4.3.10 Apache Parquet and PyArrow

Purpose: Stores raw IAM splits in a compact, columnar binary format between download and image export.

Reason for selection: Parquet is efficient for storing large image datasets with metadata columns: it compresses well, supports fast partial reads, and is the native serialization format of Hugging Face datasets. PyArrow provides the Python bindings required by

datasets for Parquet I/O. Without this intermediate format, every data preparation run would re-download from the Hub.

Usage in this project: Downloaded splits are saved as `data/iam/raw/iam_line_{split}`.

`parquet.prepare_iam_lines.py` reads these files with `load_dataset('parquet', data_files=...)` when available.

4.3.11 pandas

Purpose: Supports tabular data handling in scripts that process dataset metadata and experiment outputs (alongside raw csv module usage in core evaluation).

Reason for selection: pandas is the standard Python tool for tabular data manipulation and is a transitive dependency of datasets and matplotlib workflows. It simplifies reading, filtering, and aggregating experiment tables when generating thesis figures and filled metric documents.

Usage in this project: Used in supporting scripts (`plot_results.py`, `fill_thesis_metrics.py`) and available throughout the environment for ad hoc analysis of `predictions.csv` files.

4.3.12 PyYAML

Purpose: Defines experiment configurations in human-readable YAML files that specify run ID, model checkpoint, dataset split, batch size, sample limits, and backend type.

Reason for selection: Hard-coding experiment parameters in Python scripts makes reproduction difficult and error-prone. YAML configs separate what to run from how to run it: the same `run_evaluation()` function executes any experiment by reading a config file. This supports thesis traceability — each `experiments/<run_id>/config.json` is a frozen copy of the YAML used at run time.

Usage in this project: `configs/iam_trocr_handwritten.yaml`, `iam_demo.yaml`, and `iam_tesseract.yaml` are loaded by `scripts/run_eval.py` via `yaml.safe_load()` and mapped to an `EvalConfig` dataclass.

4.3.13 Tesseract OCR and pytesseract

Purpose: Provides a classical, non-neural OCR baseline for comparison against TrOCR on the same IAM test split.

Reason for selection: A transformer-based result is only meaningful when compared against a traditional baseline on identical data. Tesseract 5.x is the most widely available

open-source OCR engine. pytesseract is its Python wrapper, called from `evaluate.py` when `backend: tesseract` is set. If the Tesseract binary is not installed, the run is gracefully skipped and recorded in `metrics.json` with `status: skipped` — preserving pipeline reproducibility without blocking TrOCR experiments.

Usage in this project: `evaluate.py` calls `pytesseract.image_to_string(image, config='--psm 7')` for single-line mode on each IAM test image. `configs/iam_tesseract.yaml` sets `backend: tesseract`.

4.3.14 matplotlib

Purpose: Generates publication-quality bar charts comparing measured CER results against literature baselines for the thesis results chapter.

Reason for selection: Visual comparison of TrOCR, Tesseract, and cited literature CER values is required for Chapter 4 (Results). `matplotlib` is the standard Python plotting library, produces high-DPI PNG output, and requires no external GUI — suitable for headless server execution during automated experiment pipelines.

Usage in this project: `scripts/plot_results.py` reads `experiments/*/metrics.json`, builds a CER bar chart including literature anchors (AttentionHTR, Light Transformer, GFCN), and saves `experiments/figures/cer_comparison.png`.

4.3.15 setuptools and pyproject.toml (PEP 621)

Purpose: Packages the `src/htr/` library as an installable Python project with declared dependencies and version metadata.

Reason for selection: Editable installation (`pip install -e .`) ensures that scripts, the Gradio app, and evaluation CLI all import from the same `htr` package without manual `PYTHONPATH` manipulation. `pyproject.toml` is the modern standard for Python project metadata, making dependency versions explicit and reproducible for thesis examiners and future contributors.

Usage in this project: `pyproject.toml` lists all runtime dependencies. `setuptools` discovers packages under `src/` via `[tool.setuptools.packages.find]`. The project is installed once during setup.

4.3.16 JSON and CSV (standard library + flat files)

Purpose: Persists experiment metrics, per-line predictions, error summaries, and dataset labels in human-readable, tool-agnostic formats.

Reason for selection: JSON is ideal for structured metric summaries (CER, WER, n_samples) that the thesis references by file path. CSV is ideal for per-line predictions that researchers can open in Excel or pandas for qualitative review. Both formats require no database server, align with the traceability rule in docs/RESEARCH_SCOPE.md, and remain readable years later.

Usage in this project: evaluate.py writes metrics.json, predictions.csv, error_summary.json, and config.json per run.

prepare_iam_lines.py and prepare_iam_demo_examples.py write labels.csv. The Gradio header reads metrics.json for the benchmark CER summary.

4.3.17 functools.lru_cache

Purpose: Caches loaded ModelBundle instances in memory so the TrOCR model is loaded once per checkpoint and reused across Gradio requests and repeated inference calls.

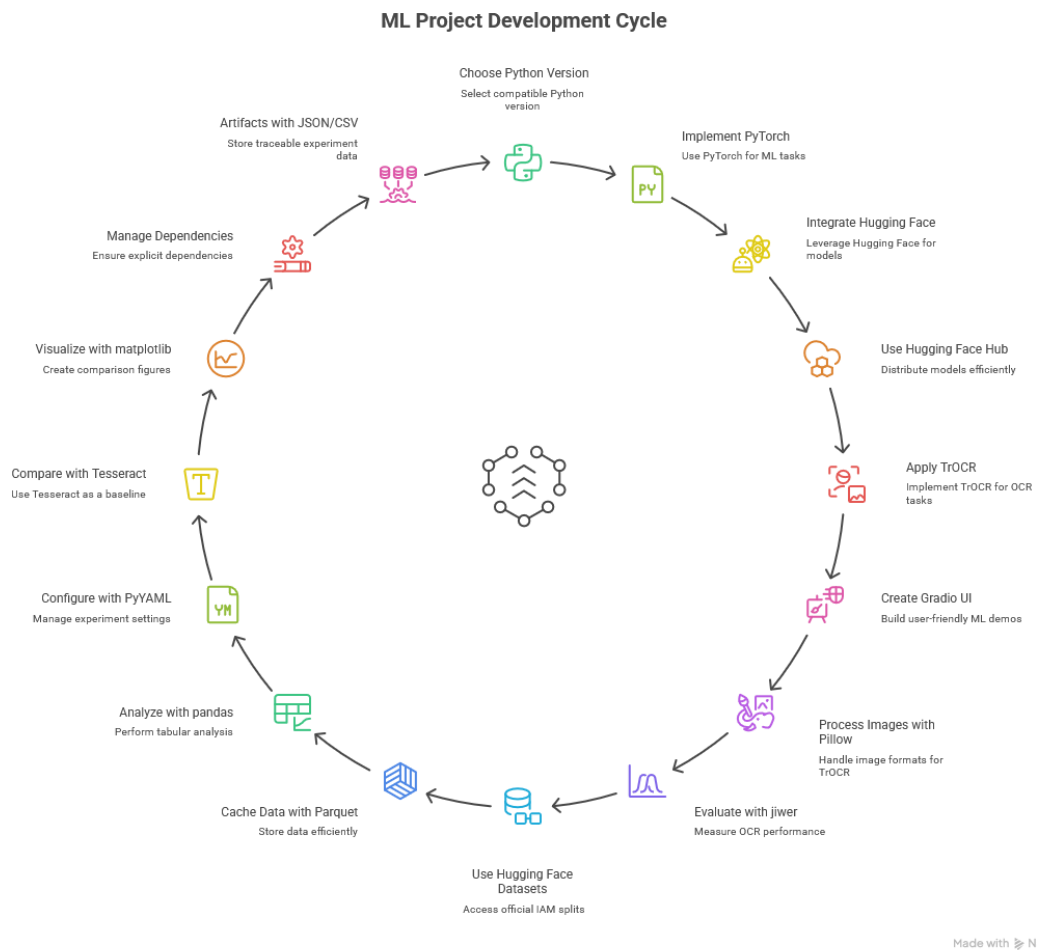
Reason for selection: Loading a transformer model from disk takes several seconds and hundreds of megabytes of RAM. Without caching, every Gradio button click would reload weights, making the demo unusable. lru_cache is a zero-dependency Python standard-library solution applied to _cached_bundle() in infer.py.

Usage in this project: recognize() calls _cached_bundle(checkpoint) which wraps load_model_bundle(). The first call loads weights; subsequent calls return the cached bundle immediately.

Technologies covered in detail

Technology	Key reason documented
Python 3.9+/3.10	ML ecosystem standard; required by all dependencies
PyTorch + torchvision	TrOCR/Hugging Face native runtime; CUDA support
Hugging Face Transformers	Official TrOCR implementation; avoids reimplementing ViT+BART
Hugging Face Hub	Model distribution without storing weights in Git
TrOCR (ViT + BART)	Thesis focus model; IAM-pretrained checkpoint
Gradio 4.x	ML demo UI without custom frontend development
Pillow	Standard image format for Gradio + TrOCRProcessor
jiwer	Dedicated CER/WER library; chosen over manual or evaluate
Hugging Face datasets	Official IAM splits from Teklia/IAM-line
Parquet + PyArrow	Efficient local caching between download and export
pandas	Tabular analysis of experiment outputs
PyYAML	Reproducible, human-readable experiment configs
Tesseract + pytesseract	Classical baseline on identical IAM data
matplotlib	Thesis CER comparison figures
setuptools / pyproject.toml	Editable install; explicit dependencies
JSON + CSV	Traceable, tool-agnostic experiment artifacts
functools.lru_cache	Avoid reloading model on every Gradio click

Table 4.1 summarizes the technology stack.



4.4 Dataset Pipeline

English IAM line-level data is sourced from the Hugging Face dataset Teklia/IAM-line, which provides official train, validation, and test splits (6,482 / 976 / 2,915 lines).

4.4.1 Download (scripts/download_iam.py)

Purpose: Fetch authoritative IAM splits from Hugging Face and cache them locally.

Reason: Ensures the same benchmark splits used in published literature. Output: Parquet files under `data/iam/raw/` and `download_meta.json` with split statistics.

```

#!/usr/bin/env python3
"""Download IAM line-level dataset from Hugging Face."""

from __future__ import annotations

import argparse
import json
from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parents[1]
RAW_DIR = REPO_ROOT / "data" / "iam" / "raw"
META_PATH = RAW_DIR / "download_meta.json"
HF_DATASET = "Teklia/IAM-line"

def download_hf_iam() -> dict:
    from datasets import load_dataset

    RAW_DIR.mkdir(parents=True, exist_ok=True)
    print(f"Loading {HF_DATASET} from Hugging Face...")
    dataset = load_dataset(HF_DATASET)

    paths = {}
    for split_name in dataset:
        hf_split = dataset[split_name]
        split_key = str(split_name)
        out_path = RAW_DIR / f"iam_line_{split_key}.parquet"
        hf_split.to_parquet(out_path)
        paths[split_key] = str(out_path)
        print(f" {split_key}: {len(hf_split)} samples -> {out_path}")

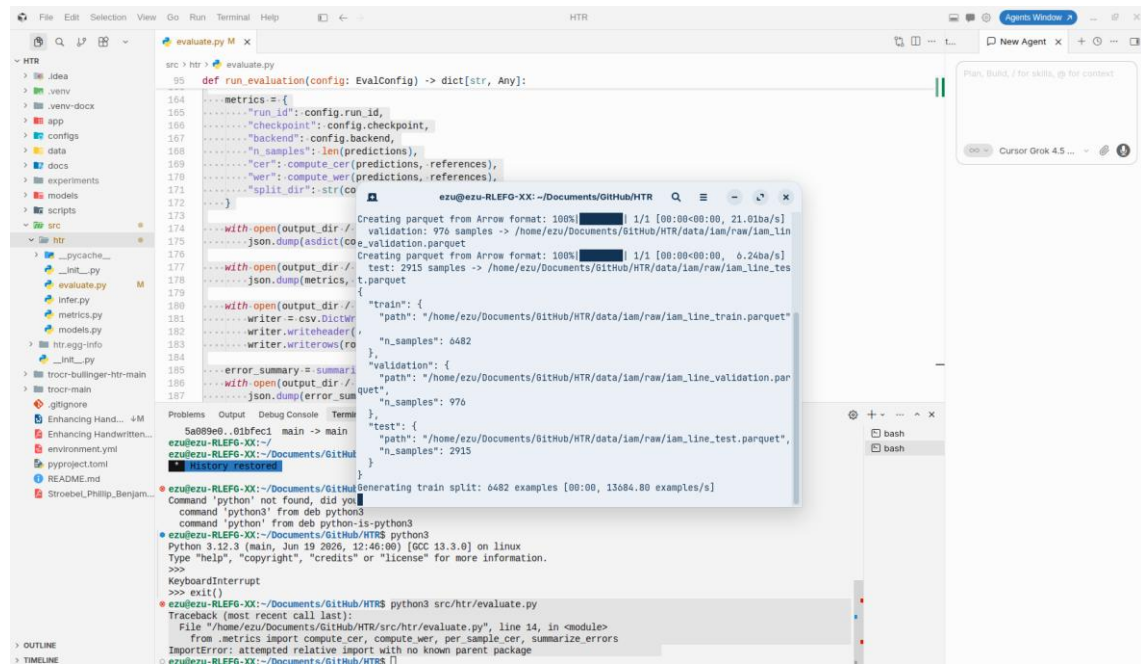
    meta = {
        "source": f"huggingface:{HF_DATASET}",
        "splits": {k: {"path": v, "n_samples": len(dataset[k])} for k, v in
paths.items()},
        "columns": dataset["train"].column_names,
    }
    META_PATH.write_text(json.dumps(meta, indent=2), encoding="utf-8")
    return meta

def main() -> None:
    parser = argparse.ArgumentParser(description="Download IAM line dataset")
    parser.parse_args()
    meta = download_hf_iam()

```

4.4.2 Export (scripts/prepare_iam_lines.py)

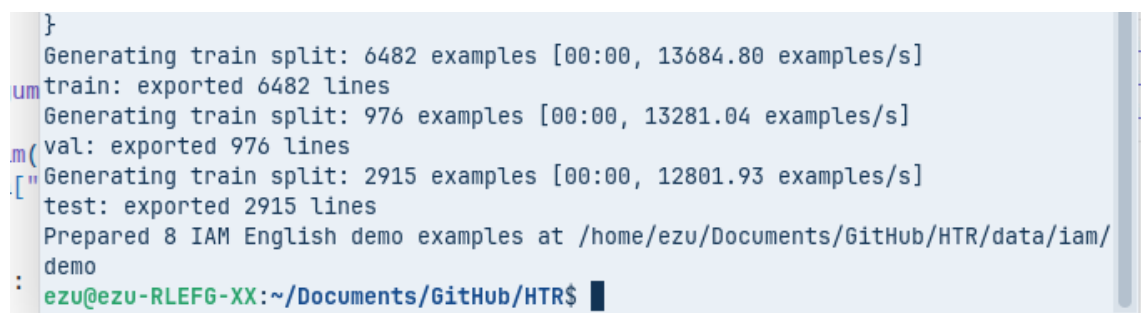
Purpose: Convert Hub/Parquet records into per-line PNG images and labels.csv manifests. Reason: The evaluation pipeline (LineDataset) expects a folder of images plus a CSV, decoupling inference from Hugging Face APIs during experiments. Output: data/iam/processed/{train,val,test}/images/ and labels.csv.



The screenshot shows a code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project structure with folders like 'data', 'scripts', and 'src'. The code editor shows the 'evaluate.py' file with a function 'def run_evaluation' that takes an 'EvalConfig' and returns a dictionary. The terminal shows the execution of the script, which creates parquet files for training, validation, and test sets, and then exports them to CSV format. The output shows the number of samples for each set: 6482 for train, 976 for validation, and 2915 for test. The script also generates a summary of the data and writes it to a file.

4.4.3 Demo Examples (scripts/prepare_iam_demo_examples.py)

Purpose: Copy eight representative English test lines into data/iam/demo/ for the Gradio Examples gallery. Reason: Gives demo users immediate sample images without downloading the full IAM dataset. Output: data/iam/demo/images/ and labels.csv.



```
}
Generating train split: 6482 examples [00:00, 13684.80 examples/s]
um train: exported 6482 lines
Generating train split: 976 examples [00:00, 13281.04 examples/s]
m val: exported 976 lines
[" Generating train split: 2915 examples [00:00, 12801.93 examples/s]
test: exported 2915 lines
Prepared 8 IAM English demo examples at /home/ezu/Documents/GitHub/HTR/data/iam/
demo
: ezu@ezu-RLEFG-XX:~/Documents/GitHub/HTR$
```

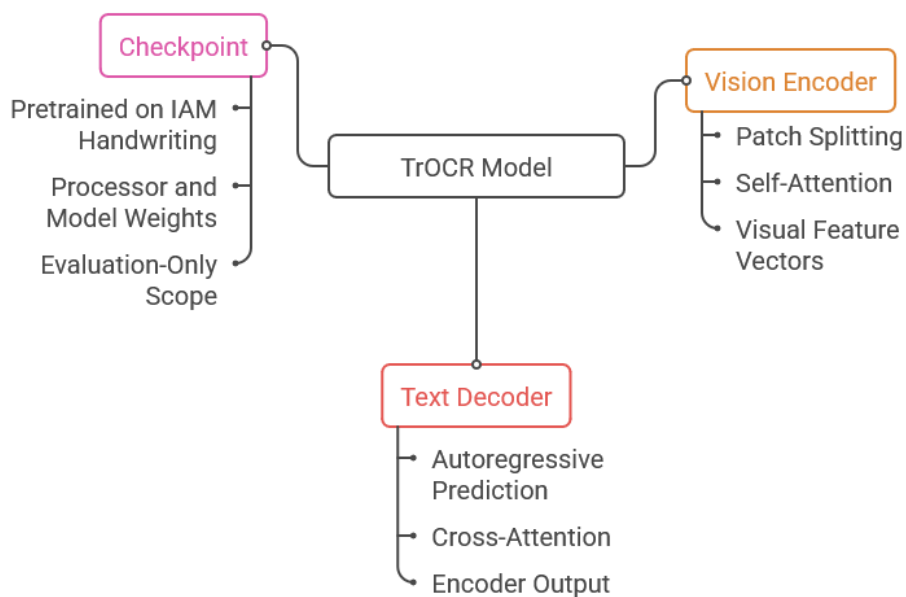
4.5 Model Architecture and Checkpoint

The system uses TrOCR (Transformer-based Optical Character Recognition), consisting of:

1. Vision Encoder (ViT): Splits the line image into patches, applies self-attention, and produces a sequence of visual feature vectors.
2. Text Decoder (BART): Autoregressively predicts the next character/token given visual features and previously generated tokens, using cross-attention to the encoder output.

The checkpoint `microsoft/trocr-base-handwritten` is pretrained on English IAM handwriting. Processor and model weights are loaded as a matched pair from the same checkpoint. Evaluation-only scope: no fine-tuning is performed in this thesis; the pretrained weights are used directly, consistent with `docs/RESEARCH_SCOPE.md`.

TrOCR Model Architecture and Checkpoint



Made with  Napkin

4.6 Core Library Components

4.6.1 Model Loading — `models.py`

Purpose: Centralize checkpoint validation, processor/model loading, device placement, and batched generation. Reason: Prevents the Gradio app and evaluation pipeline from duplicating Hugging Face boilerplate.

```
resolved_device = device or DEFAULT_DEVICE
processor = TrOCRProcessor.from_pretrained(checkpoint)
model = VisionEncoderDecoderModel.from_pretrained(checkpoint)
model.to(resolved_device)
model.eval()
```

ModelBundle dataclass exposes `preprocess()`, `generate()`, and `generate_batch()` methods. KNOWN_CHECKPOINTS restricts loading to vetted checkpoints only.

4.6.2 Inference — `infer.py`

Purpose: Provide a single-call `recognize(image, checkpoint)` API for scripts and the Gradio app. Reason: Decouples UI event handlers from model internals.

```
@lru_cache(maxsize=4)
def _cached_bundle(checkpoint: str) -> ModelBundle:
    return load_model_bundle(checkpoint)

def _load_image(image: ImageInput) -> Image.Image:
    if isinstance(image, str):
        return Image.open(image).convert("RGB")
    return image.convert("RGB")

def recognize(image: ImageInput, checkpoint: str) -> str:
    bundle = _cached_bundle(checkpoint)
    pil_image = _load_image(image)
    return normalize_text(bundle.generate(pil_image))
```

4.6.3 Metrics — `metrics.py`

Purpose: Implement thesis-aligned CER/WER and error summarization. Reason: One metrics module guarantees that benchmark numbers and live Gradio CER use identical normalization.

```
def normalize_text(text: str) -> str:
    """Case-sensitive normalization: collapse internal whitespace, strip ends."""
    return re.sub(r"\s+", " ", text.strip())

def compute_cer(predictions: list[str], references: list[str]) -> float:
    preds = [normalize_text(p) for p in predictions]
    refs = [normalize_text(r) for r in references]
    return float(jiwer.cer(refs, preds))
```

4.6.4 Batch Evaluation — evaluate.py

Purpose: Run reproducible corpus-level experiments from YAML configs with trocr or tesseract backends. Reason: Separates one-off demo inference from systematic benchmark evaluation with frozen artifacts.

Invocation:

```
scripts/run_eval. python3 py --config configs/iam_trocr_handwritten.yaml
```

```
#!/usr/bin/env python3
"""Run HTR evaluation from a YAML config."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

import yaml

REPO_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(REPO_ROOT / "src"))

from htr.evaluate import EvalConfig, run_evaluation


def load_config(path: Path) -> EvalConfig:
    with open(path, encoding="utf-8") as f:
        raw = yaml.safe_load(f)

    split_dir = REPO_ROOT / raw["split_dir"]
    output_dir = REPO_ROOT / raw.get("output_dir", "experiments")

    return EvalConfig(
        run_id=raw["run_id"],
        checkpoint=raw.get("checkpoint", ""),
        split_dir=split_dir,
        output_dir=output_dir,
        batch_size=int(raw.get("batch_size", 8)),
        max_samples=raw.get("max_samples"),
        backend=raw.get("backend", "trocr"),
        seed=int(raw.get("seed", 42)),
    )


def main() -> None:
    parser = argparse.ArgumentParser(description="Run HTR evaluation")
    parser.add_argument("--config", required=True, help="Path to YAML config")
    args = parser.parse_args()

    config_path = Path(args.config)
    if not config_path.is_absolute():
        config_path = REPO_ROOT / config_path
```

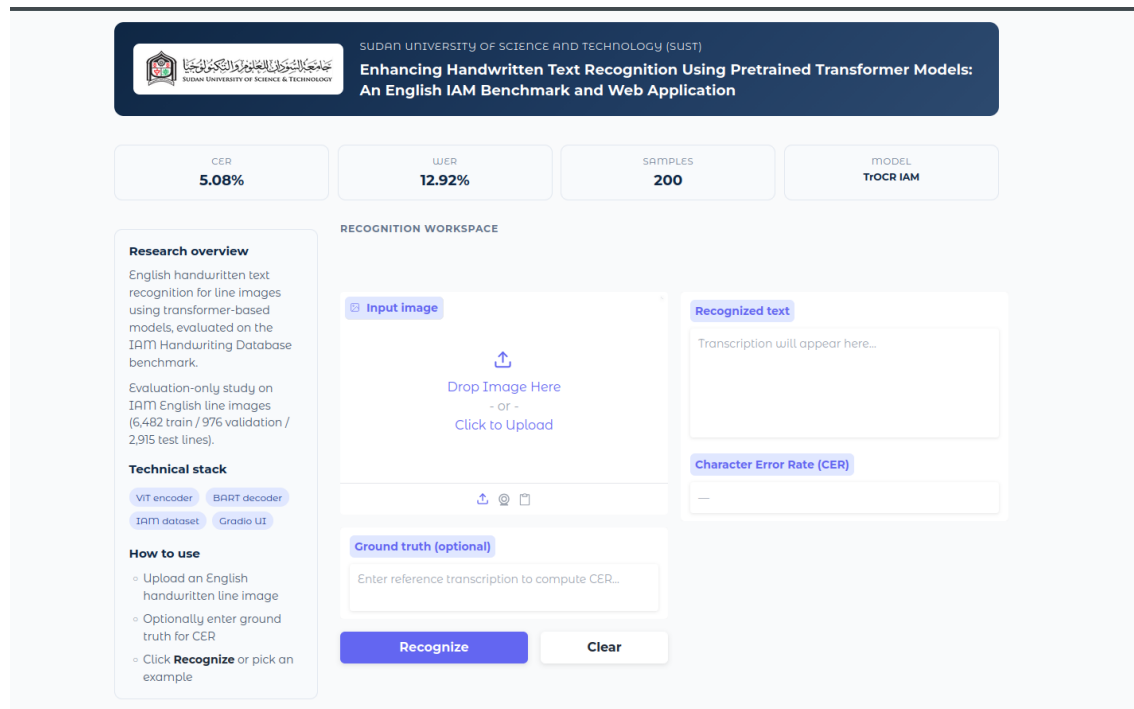
Each run produces metrics.json, predictions.csv, error_summary.json, and config.json.

4.7 Gradio Application (app/gradio_app.py)

Purpose: Demonstrate the research model to non-technical users via a browser. Reason: Satisfies thesis objective 5 (deploy a web application) while reusing src/htr/ for identical inference behaviour.

4.7.1 Interface Structure

- Header: Title, architecture summary, and IAM benchmark CER (from experiments/ if available)
- Input panel: Image upload (PIL), optional ground-truth textbox, Recognize and Clear buttons
- Output panel: Recognized text and per-line CER when ground truth is supplied
- Examples gallery: Eight IAM English demo lines from data/iam/demo/
- Footer: Model and dataset attribution



4.7.2 Recognition Handler

```
def process_image(image, ground_truth: str):
    if image is None:
        return "", ""
    text = recognize(image, TROCR_HANDWRITTEN)
    if ground_truth and ground_truth.strip():
        cer = compute_cer([text], [ground_truth])
        return text, f"{cer:.4f}"
    return text, "Ground truth not provided"
```

4.8 Experiment Configuration

configs/iam_trocr_handwritten.yaml — primary TrOCR result on IAM test (default 200 lines)

```
run_id: iam_trocr_handwritten
checkpoint: microsoft/trocr-base-handwritten
split_dir: data/iam/processed/test
output_dir: experiments
batch_size: 8
max_samples: 200
backend: trocr
seed: 42
```

configs/iam_demo.yaml — qualitative demo subset (8 lines)

```
run_id: iam_demo
checkpoint: microsoft/trocr-base-handwritten
split_dir: data/iam/demo
output_dir: experiments
batch_size: 4
backend: trocr
seed: 42
```

configs/iam_tesseract.yaml — Tesseract baseline on the same IAM test split

```
run_id: iam_tesseract
checkpoint: tesseract
split_dir: data/iam/processed/test
output_dir: experiments
batch_size: 1
backend: tesseract
seed: 42
```

max_samples: 200 keeps CPU-only evaluation tractable. Removing it evaluates all 2,915 test lines on a GPU machine. All outputs land in experiments/<run_id>/.

4.9 Evaluation Metrics and Traceability

Case-sensitive CER and WER (jiwer, after whitespace normalization)

No lowercasing of transcriptions

Corpus CER = total character edits / total reference characters

Every thesis number references experiments/<run_id>/metrics.json

scripts/plot_results.py and scripts/fill_thesis_metrics.py connect measured numbers to thesis prose and figures.

☐ Input image

Recognized text

I don't think he will storm the charts with this one , but it's a good start .

Character Error Rate (CER)

0.0127

Ground truth (optional)

I don't think he will storm the charts with this one , but it 's a good start .

Recognize

Clear

4.9.1 IAM primary result

Metric	Value
CER	5.08%
WER	12.92%
n	200

Artifact: `experiments/iam_trocr_handwritten/predictions.csv`

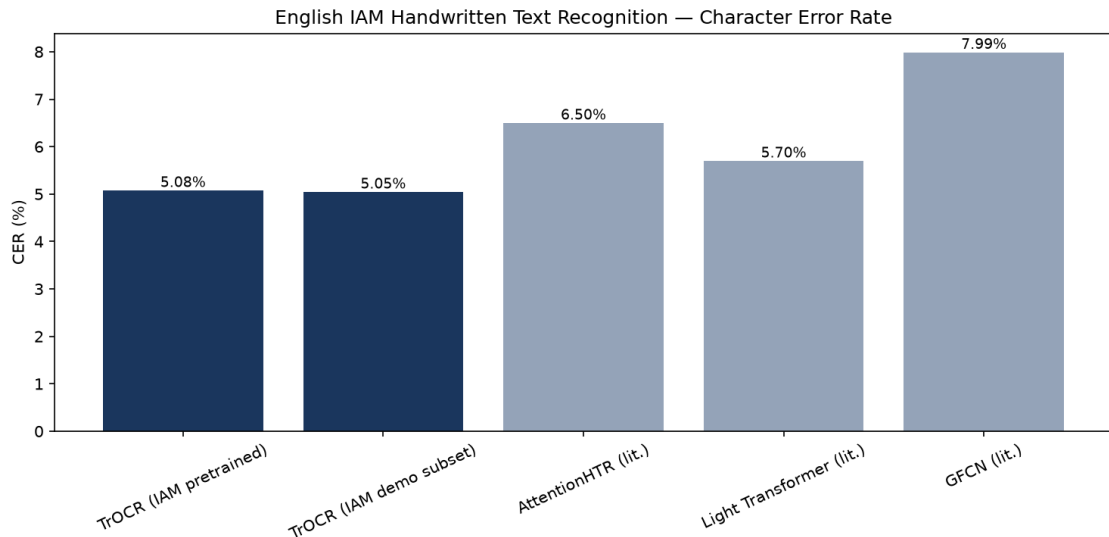
4.9.2 Literature comparison

Method	CER (IAM)	Source
TrOCR (this work)	5.08%	experiments/
AttentionHTR	6.50%	Kass & Vats, 2022
Light Transformer	5.70%	Barrère et al., 2022
GFCN	7.99%	Coquenat et al., 2020

4.9.3 IAM Qualitative (`iam_demo`)

Metric	Value
CER	5.05%
WER	14.97%
n	8

See `experiments/iam_demo/predictions.csv`



4.11 Performance Considerations

- ✓ LRU-cached ModelBundle: avoids per-request model reload in the Gradio app
- ✓ Batched evaluation (default batch_size=8): amortizes GPU kernel launch overhead
- ✓ Automatic CUDA/CPU device selection: runs on laptops and servers alike
- ✓ RGB preprocessing via TrOCRProcessor: no extra binarization latency
- ✓ max_new_tokens=128: bounds decoder runtime per line
- ✓ Graceful degradation: missing Tesseract or demo data yields informative messages, not crashes.

